

LMS API Endpoint Documentation

Base URL: `/api/v1` **Auth:** Bearer token (JWT) on every request unless marked *Public*. Use `/auth/refresh-token` to get a new access token when it expires. **Roles:** Admin, Lecturer, Student — shown per endpoint. Admin generally has access to everything a Lecturer or Student has, so it's only called out when a rule is Lecturer- or Student-specific (e.g. "only assigned courses", "self").

This document merges two team drafts (the general LMS endpoint list and the Family Medicine Forum LMS reference) into a single spec. It follows the structure/formatting of the first draft, and folds in every additional module from the second draft (Question Bank, Exams, Assignments, Recorrection, Attendance, Progress Tracking, Notifications, Payments) so nothing from either version is lost.

1. Auth & Users

Method	Endpoint	Description	Access
POST	<code>/auth/register</code>	Register new user (used internally when Admin registers a student or lecturer)	Admin
POST	<code>/auth/login</code>	Login, returns JWT + refresh token	Public
POST	<code>/auth/refresh-token</code>	Exchange a valid refresh token for a new access token	Public
POST	<code>/auth/logout</code>	Invalidate session/token	All
POST	<code>/auth/forgot-password</code>	Trigger password reset email	Public
POST	<code>/auth/reset-password</code>	Reset password with token	Public
GET	<code>/users/me</code>	Get current logged-in user profile	All
PUT	<code>/users/me</code>	Update own profile	All
PUT	<code>/users/me/profile-image</code>	Upload/update own profile image	All
GET	<code>/users</code>	List all users (filterable by role, status)	Admin
GET	<code>/users/:user_id</code>	Get any user's basic info	Admin
PUT	<code>/users/:user_id</code>	Update a user's details	Admin
PUT	<code>/users/:user_id/status</code>	Activate / suspend a user	Admin
DELETE	<code>/users/:user_id</code>	Delete a user	Admin

2. Student Management

Method	Endpoint	Description	Access
POST	/students	Register new student (creates users + students row)	Admin
GET	/students	List all students (paginated, filterable by course/batch/status)	Admin, Lecturer*
GET	/students/:student_id	View a student's full profile	Admin, Lecturer
PUT	/students/:student_id	Update student info	Admin
DELETE	/students/:student_id	Delete a student	Admin
GET	/students/:student_id/enrollments	View all courses/batches a student is enrolled in	Admin, Lecturer, Student (self)
GET	/students/:student_id/progress	View a student's module-level progress	Admin, Lecturer, Student (self)
GET	/students/:student_id/payments	View a student's payment history	Admin, Student (self)

* Lecturer results restricted to students in their assigned batches (see §5 Lecturer Assignments).

3. Instructor / Lecturer Management

Method	Endpoint	Description	Access
POST	/lecturers	Add new lecturer	Admin
GET	/lecturers	View all lecturers	Admin
GET	/lecturers/:instructor_id	View a lecturer's profile	Admin
PUT	/lecturers/:instructor_id	Update lecturer info (incl. bio/specialization)	Admin
DELETE	/lecturers/:instructor_id	Delete a lecturer	Admin
GET	/lecturers/:instructor_id/assignments	View courses/batches assigned to a lecturer	Admin, Lecturer (self)

4. Course Management

Method	Endpoint	Description	Access
POST	/courses	Add new course	Admin
GET	/courses	List all courses	Admin, Lecturer, Student*
GET	/courses/:course_id	View course details	Admin, Lecturer, Student
PUT	/courses/:course_id	Update course info	Admin, Lecturer (only assigned)
DELETE	/courses/:course_id	Delete course	Admin

* Lecturer and Student results restricted to courses they're assigned to / enrolled in.

5. Batch Management & Lecturer Assignments

Method	Endpoint	Description	Access
POST	/courses/:course_id/batches	Add new batch for a course	Admin
GET	/courses/:course_id/batches	View course-wise batches	Admin, Lecturer, Student
GET	/batches	View all batches (supports <code>?course_id=&status=&start_date=&end_date=</code>)	Admin, Lecturer*
GET	/batches/:batch_id	View batch details	Admin, Lecturer, Student
PUT	/batches/:batch_id	Update batch info	Admin
PUT	/batches/:batch_id/dates	Update start/end dates	Admin
PUT	/batches/:batch_id/access	Grant/block LMS access for entire batch	Admin
POST	/batches/:batch_id/lecturers	Assign a lecturer to a batch	Admin
DELETE	/batches/:batch_id/lecturers/:instructor_id	Unassign a lecturer from a batch	Admin

* Lecturer sees only batches they're assigned to.

6. Enrollment & Access Control

Method	Endpoint	Description	Access
POST	/batches/:batch_id/enrollments	Enroll a student in a batch	Admin
GET	/batches/:batch_id/enrollments	List students enrolled in a batch	Admin, Lecturer
PUT	/enrollments/:enrollment_id	Update enrollment status (e.g. active / completed / dropped)	Admin
DELETE	/enrollments/:enrollment_id	Remove a student from a batch	Admin
PUT	/enrollments/:enrollment_id/access	Grant/block LMS access for one student	Admin

7. Module & Material Management

Method	Endpoint	Description	Access
POST	/courses/:course_id/modules	Add module to a course	Admin
GET	/courses/:course_id/modules	List modules of a course	Admin, Lecturer, Student
PUT	/modules/:module_id	Update module info / reorder	Admin
DELETE	/modules/:module_id	Delete module	Admin
POST	/modules/:module_id/materials	Add new material (course-level; recording or PDF)	Admin, Lecturer (assigned)
POST	/batches/:batch_id/modules/:module_id/materials	Add batch-specific material	Admin, Lecturer (assigned)
GET	/modules/:module_id/materials	List materials in a module	Admin, Lecturer, Student
GET	/batches/:batch_id/materials	List materials for a specific batch	Admin, Lecturer, Student
GET	/materials/:material_id	Get material details / file URL	Admin, Lecturer, Student
PUT	/materials/:material_id	Modify material (creates new version for PDFs)	Admin, Lecturer (assigned)
DELETE	/materials/:material_id	Delete / archive material	Admin, Lecturer (assigned)
PUT	/materials/:material_id/visibility	Access control — show/hide material	Admin, Lecturer (assigned)
PUT	/materials/:material_id/deadline	Set/update deadline for a material	Admin, Lecturer (assigned)
PUT	/materials/:material_id/progress	Student updates own watch/read progress	Student

Method	Endpoint	Description	Access
GET	/materials/:material_id/progress	View all students' progress on a material	Admin, Lecturer
GET	/students/:student_id/materials/:material_id/progress	View one student's progress on one material	Admin, Lecturer, Student (self)

8. Question Bank Management

Method	Endpoint	Description	Access
POST	/questions	Add new question	Admin, Lecturer (assigned course)
GET	/courses/:course_id/questions	List questions for a course	Admin, Lecturer
GET	/questions/:question_id	Question detail (with options)	Admin, Lecturer
PUT	/questions/:question_id	Edit/update question	Admin, Lecturer (assigned)
DELETE	/questions/:question_id	Remove question	Admin, Lecturer (assigned)
POST	/questions/:question_id/options	Add option to a question	Admin, Lecturer
PUT	/options/:option_id	Update option / mark correct	Admin, Lecturer
DELETE	/options/:option_id	Delete option	Admin, Lecturer

9. Exam Management

Method	Endpoint	Description	Access
POST	/batches/:batch_id/exams	Add new exam for a batch	Admin, Lecturer (assigned)
GET	/batches/:batch_id/exams	List exams for a batch	Admin, Lecturer, Student
GET	/exams/:exam_id	Exam details	Admin, Lecturer, Student
PUT	/exams/:exam_id	Update exam	Admin, Lecturer (assigned)
DELETE	/exams/:exam_id	Delete exam	Admin, Lecturer (assigned)
POST	/exams/:exam_id/questions	Attach question(s) to exam	Admin, Lecturer
GET	/exams/:exam_id/questions	List exam questions	Admin, Lecturer, Student (during attempt)
DELETE	/exams/:exam_id/questions/:question_id	Remove question from exam	Admin, Lecturer
POST	/exams/:exam_id/eligibility/auto-check	Run automatic eligibility check	Admin, Lecturer
PUT	/exams/:exam_id/eligibility	Bulk manual set eligibility (1+ students)	Admin, Lecturer
GET	/exams/:exam_id/eligibility	View eligibility list	Admin, Lecturer
POST	/exams/:exam_id/attempts	Start exam attempt	Student
GET	/attempts/:attempt_id	Attempt detail	Admin, Lecturer, Student (self)
PUT	/attempts/:attempt_id/submit	Submit exam attempt	Student
GET	/exams/:exam_id/attempts	List all attempts for an exam	Admin, Lecturer

Method	Endpoint	Description	Access
POST	/attempts/:attempt_id/answers	Submit answer(s) during attempt	Student
GET	/attempts/:attempt_id/answers	View answers for an attempt	Admin, Lecturer, Student (self)
PUT	/answers/:answer_id/grade	Manually grade/update marks (e.g. text answers)	Admin, Lecturer
GET	/exams/:exam_id/results	Exam result summary (all students)	Admin, Lecturer
GET	/students/:student_id/exams/:exam_id/result	Individual result	Admin, Lecturer, Student (self)
GET	/exams/:exam_id/live-stream	Get live stream URL (if applicable)	Admin, Lecturer, Student

10. Assignment Management

Method	Endpoint	Description	Access
POST	/batches/:batch_id/assignments	Set assignment for a batch	Admin, Lecturer (assigned)
GET	/batches/:batch_id/assignments	List assignments (Student view includes notification flag)	Admin, Lecturer, Student
GET	/assignments/:assignment_id	Assignment detail	Admin, Lecturer, Student
PUT	/assignments/:assignment_id	Update assignment	Admin, Lecturer
DELETE	/assignments/:assignment_id	Delete assignment	Admin, Lecturer
POST	/assignments/:assignment_id/submissions	Upload/submit answer file	Student
GET	/assignments/:assignment_id/submissions	List submissions for an assignment	Admin, Lecturer
GET	/submissions/:submission_id	View a submission / file	Admin, Lecturer, Student (self)
PUT	/submissions/:submission_id/grade	Mark/grade + feedback	Admin, Lecturer
GET	/students/:student_id/submissions	All submissions by a student	Admin, Lecturer, Student (self)

11. Recorrection Requests

Method	Endpoint	Description	Access
POST	/recorrection-requests	Raise a recorrection request (exam or assignment)	Student
GET	/recorrection-requests	List requests (filterable by status/type)	Admin, Lecturer
GET	/recorrection-requests/:request_id	Request detail	Admin, Lecturer, Student (self)
PUT	/recorrection-requests/:request_id/resolve	Resolve with response note	Admin, Lecturer
GET	/students/:student_id/recorrection-requests	Student's request history	Admin, Lecturer, Student (self)

12. Attendance Management

Method	Endpoint	Description	Access
POST	/batches/:batch_id/attendance-sessions	Create attendance session	Admin, Lecturer
GET	/batches/:batch_id/attendance-sessions	List sessions for a batch	Admin, Lecturer
GET	/attendance-sessions/:session_id	Session detail	Admin, Lecturer
POST	/attendance-sessions/:session_id/records	Mark attendance (bulk)	Admin, Lecturer
GET	/attendance-sessions/:session_id/records	View records for a session	Admin, Lecturer
PUT	/attendance-records/:record_id	Correct a single record	Admin, Lecturer
GET	/students/:student_id/attendance	Student attendance summary	Admin, Lecturer, Student (self)

13. Progress Tracking (Module-Level)

Material-level progress endpoints live in §7 Module & Material Management. [GET](#)

[/students/:student_id/progress](#) (module-level summary) is listed in §2 Student Management — the two

endpoints below extend it.

Method	Endpoint	Description	Access
GET	/modules/:module_id/progress	All students' progress in a module	Admin, Lecturer
PUT	/students/:student_id/modules/:module_id/progress/status	Update completion %	Student (own, auto-tracked), Admin, Lecturer

14. Notification Management

Method	Endpoint	Description	Access
POST	/notifications	Create/send notification (target: batch, course, or user)	Admin, Lecturer
GET	/notifications	List sent notifications	Admin, Lecturer
GET	/notifications/:notification_id	Notification detail	Admin, Lecturer
GET	/users/:user_id/notifications	Inbox for a user	Admin, Lecturer, Student (self)
PUT	/notifications/:notification_id/read	Mark as read	Admin, Lecturer, Student (self)
DELETE	/notifications/:notification_id	Delete notification	Admin, Lecturer

15. Payment Management

Method	Endpoint	Description	Access
POST	/payments	Record a payment	Admin
GET	/payments	List payments (supports <code>?course_id=&batch_id=&student_id=</code> filters)	Admin
GET	/payments/:payment_id	Payment detail	Admin
PUT	/payments/:payment_id	Update payment status/reference	Admin
GET	/payments/analytics	Aggregated payment analytics (dashboard)	Admin
POST	/payment-plans	Create payment plan for a course	Admin
GET	/payment-plans	List plans (filterable by course)	Admin
PUT	/payment-plans/:plan_id	Update plan	Admin
DELETE	/payment-plans/:plan_id	Delete plan	Admin
POST	/payment-installments	Generate installments for an enrollment	Admin
GET	/enrollments/:enrollment_id/installments	List installments for an enrollment	Admin, Student (self)
PUT	/installments/:installment_id	Update installment status/payment link	Admin
PUT	/installments/:installment_id/due-date	Update installment due date	Admin

(See also `GET /students/:student_id/payments` in §2 Student Management for a student's payment history.)

Implementation Notes

Route structure: use nested resource routes (e.g. `/courses/:course_id/batches`) for listing-in-context, and flat routes (`/batches/:batch_id`) for direct CRUD — this matches the FK relationships in the schema.

Middleware order: `authenticate` (JWT) → `authorize(['admin', 'lecturer'])` → optional `checkAssignment` (verifies a lecturer's assignment row before allowing writes to a course/batch/module/material/exam/assignment they don't own).

File uploads (profile image, material/assignment/submission files): handle via multer + a separate `/uploads` route, or a direct-to-S3 pre-signed URL pattern; store only the resulting URL in the DB.

Bulk eligibility: `PUT /exams/:exam_id/eligibility` should accept an array of `student_ids` + `is_eligible` + `method` to match the "select 1 or more students" requirement.

Status-only updates: most tables have a `status` column. This document standardizes on `PUT` for both full and status-only updates to match the original structure — teams that prefer stricter REST semantics can swap these specific routes to `PATCH` without changing the URL shape.